Bases de données
SQL

Expressions
Sélections spécialisées

SQL_03c
v312f

2025-02-11

Christina.Khnaisser@USherbrooke.ca
Luc.Lavoie@USherbrooke.ca

© 2018-2021, **Μῆτις** (http://info.usherbrooke.ca/llavoie)
CC BY-NC-SA 4.0 (https://creativecommons.org/licenses/by-nc-sa/4.0/)

Expressions relationnelles en SQL

- Rappels
- SELECT
  - Agrégation
  - Groupement
  - Quantification
  - Ordonnancement
  - Divers
- Exemples

Le Langage SQL
SELECT (rappels)

*Les **catégories en vert**... nous y sommes!*

requête ::=

[*Contexte*]
SELECT *opMode* { * | *projection-extension* }
FROM *listeDeJointures*
[*Restriction*]
[***Groupement***]
[*OpComplémentaire*]
[***Ordonnancement***]
[***Divers***]

opMode ::=

[DISTINCT | ALL]

2025-02-11 MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, M.Frappier — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

[WITH [RECURSIVE] *requête_with* [, ...]]
SELECT [ALL | DISTINCT [ON (*expression* [, ...])]]
* | *expression* [[AS] *nom_d_affichage*] [, ...]
[FROM *éléments_from* [, ...]]
[WHERE *condition*]
[GROUP BY *expression* [, ...]]
[HAVING *condition* [, ...]]
[WINDOW *nom_window* AS (*définition_window*) [, ...]]
[{ UNION | INTERSECT | EXCEPT } [ALL | DISTINCT] *select*]
[ORDER BY *expression* [ASC | DESC | USING *opérateur*] [NULLS { FIRST | LAST }] [, ...]]
[LIMIT { *nombre* | ALL }]
[OFFSET *début*] [ROW | ROWS]]
[FETCH { FIRST | NEXT } [*total*] { ROW | ROWS } ONLY]
[FOR { UPDATE | SHARE } [OF *nom_table* [, ...]] [NOWAIT] [...]]

avec *éléments_from* qui peut être :

[ONLY] *nom_table* [*] [[AS] *alias* [(*alias_colonne* [, ...])]]
(*select*) [AS] *alias* [(*alias_colonne* [, ...])]
nom_requête_with [[AS] *alias* [(*alias_colonne* [, ...])]]
nom_fonction ([*argument* [, ...]]) [AS] *alias* [(*alias_colonne* [, ...] | *définition_colonne* [, ...])]

nom_fonction ([*argument* [, ...]]) AS (*définition_colonne* [, ...])
éléments_from [NATURAL] *type_jointure éléments_from* [ON *condition_jointure* | USING (*colonne_jointure* [, ...])]

et *requête_with* est :

nom_requête_with [(*nom_colonne* [, ...])] AS (*select* | *valeurs* | *insert* | *update* | *delete*)

TABLE [ONLY] *nom_table* [*]

Le Langage SQL
Fonctions d'agrégation (échantillon)

- ***Fonctions de base***
  - AVG(*col*)
  - COUNT(*col*)
  - MAX(*col*)
  - MEDIAN(*col*)
  - MIN(*col*)
  - STDDEV(*col*)
    - STDDEV_POP(*col*)
    - STDDEV_SAMP(*col*)
  - SUM(*col*)
  - VARIANCE(*col*)
    - VAR_POP(*col*)
    - VAR_SAMP(*col*)

- ***Fonctions logiques***
  - EVERY(*col*) -> bool_and(*col*)
  - ANY(*col*) -> bool_or(*col*)

- *col* ::= [ALL|DISTINCT] *nomCol*

- Quand mettre ALL ou DISTINCT ?
- Que se passe-t-il s'il n'y a aucun tuple ?
- Comment les NULL sont-ils traités ?
- Un cas spécial : COUNT(*)

2025-02-11 MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mjftq — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

1) If, during the computation of the result of *AF,* an intermediate result is not representable in the declared type of the site that contains that intermediate result, then

Case:

a) If the most specific type of the result of *AF* is an interval type, then an exception condition is raised:
data exception — interval value out of range.

b) If the most specific type of the result of *AF* is a multiset type, then an exception condition is raised:
data exception — multiset value overflow.

c) Otherwise, an exception condition is raised: *data exception — numeric value out of range.*

2) Case:

a) If <filter clause> is specified, then the <search condition> is applied to each row of *T.* Let *T1* be the collection of rows of *T* for which the result of the <search condition> is *True.*

b) Otherwise, let *T1* be *T.*

3) If COUNT(*) is specified, then the result is the cardinality of *T1.*

4) If <general set function> is specified, then:

a) Let *TX* be the single-column table that is the result of applying the <value expression> to each row of *T1* and eliminating null values. If one or more null values are eliminated, then a completion condition is raised: *warning — null value eliminated in set function.*

b) Case:

i) If DISTINCT is specified, then let *TXA* be the result of eliminating redundant duplicate values from *TX,* using the comparison rules specified in Subclause 8.2, “<comparison predicate>”, to identify the redundant duplicate values.

ii) Otherwise, let *TXA* be *TX.*

c) Let *N* be the cardinality of *TXA*

d) Case:

i) If COUNT is specified, then the result is *N.*

ii) If *TXA* is empty, then the result is the null value.

iii) If AVG is specified, then the result is the average of the values in *TXA.*

iv) If MAX or MIN is specified, then the result is respectively the maximum or minimum value in *TXA.* These results are determined using the comparison rules specified in Subclause 8.2, “<comparison predicate>”. If *DT* is a user-defined type and the comparison of two values in *TXA* results in *Unknown,* then the maximum or minimum of *TXA* is implementation-dependent.

v) If SUM is specified, then the result is the sum of the values in *TXA.* If the sum is not within the range of the declared type of the result, then an exception condition is raised: *data exception — numeric value out of range.*

vi) If EVERY is specified, then Case:

1) If the value of some element of *TXA* is *False,* then the result is *False.*
2) Otherwise, the result is *True.*

vii) If ANY or SOME is specified, then
Case:

1) If the value of some element of *TXA* is *True,* then the result is *True.*
2) Otherwise, the result is *False.*

viii) If VAR_POP or VAR_SAMP is specified, then let *S1* be the sum of values in the column of *TXA,* and *S2* be the sum of the squares of the values in the column of *TXA.*

1) If VAR_POP is specified, then the result is (S2-S1*S1/N)/N.
2) If VAR_SAMP is specified, then
Case:

A) If *N* is 1 (one), then the result is the null value.

B) Otherwise, the result is (S2-S1*S1/N)/(N-1)

ix) If FUSION is specified, then the result is the multiset *M* such that for each value *V* in the element type of *DT,* including the null value, the number of elements of *M* that are identical to *V* is the sum of the number of identical copies of *V* in the multisets that are the values of the column in each row of *TXA.*

x) If INTERSECTION is specified, then the result is a multiset *M* such that for each value *V* in the element type of *DT,* including the null value, the number of duplicates of *V* in *M* is the minimum of the number of duplicates of *V* in the multisets that are the values of the column in each row of *TXA.*

NOTE 231 — This rule says “the result is a multiset” rather than “the result is the multiset” because the precise duplicate values are not specified. Thus this calculation is non-deterministic for certain element types, namely those based on character

string, datetime with time zone and user-defined types.

Le Langage SQL
Fonctions d'agrégation (bis)

- Plusieurs autres disponibles, dont
  - string_agg, xml_agg, jason_agg
  - array_agg, range_agg
  - bit_or, bit_and
  - de très nombreuses fonctions statistiques

- Voir
  - https://docs.postgresql.fr/17/functions-aggregate.html

Évaluation – Exemple de données (AGG)

Activité

sigle	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

TypeÉvaluation

code	description
IN	Examen intra
FI	Examen final
TP	Travail pratique
PR	Projet

Résultat

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20133	80
15112354	FI	IFT187	20123	78
15113150	TP	IFT159	20133	75
15112354	FI	GMQ103	20123	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

Étudiant

matricule	nom	adresse
15113150	Paul	>ᕕᖕᓂᑐᖅ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

2025-02-11 MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mᵃᵗᵗᵃˢ — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Calculer la moyenne des résultats
Calculer la moyenne des résultats des examens (IN, FI) de l'activité IFT187; le minimum, le maximum...

Groupement

Groupement ::=

[GROUP BY { *expression* ... , }]

[HAVING { *condition* ... , }]

- beaucoup de choses à dire ☺
- ... et à expérimenter

MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mffps — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Évaluation – Exemple de données (GROUP)

Activité

sigle	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

TypeÉvaluation

code	description
IN	Examen intra
FI	Examen final
TP	Travail pratique
PR	Projet

Résultat

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20133	80
15112354	FI	IFT187	20123	78
15113150	TP	IFT159	20133	75
15112354	FI	GMQ103	20123	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

Étudiant

matricule	nom	adresse
15113150	Paul	>ᕕᖕᓂᑐᖅ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

2025-02-11 MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mᵃᵗᵗᵉᵒ — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Le nombre d'évaluations consignées par étudiant :

SELECT matricule, COUNT(*) AS nbEvaluations
FROM resultat
GROUP BY matricule

Le nombre d'activités auxquelles un étudiant est ou a été inscrit :

SELECT matricule, COUNT(DISTINCT activite) AS nbActivites
FROM resultat
GROUP BY matricule

Le nombre de trimestrestres distincts au cours desquels un une activité pédagogique a été offerte

SELECT activite, *COUNT*(DISTINCT trimestre) AS nbFois
FROM resultat
GROUP BY activite

Note totale par étudiant pour chacun de ces cours

SELECT matricule, activite, *SUM*(note) AS noteTotale
FROM resultat
GROUP BY matricule, activite

Quantification

- Motivation
- Quantification générale
- Quantification de comparaison

Quantification
Motivation

- Compléter l'équivalence avec la logique du premier ordre.
- Caractériser les cas où il est nécessaire d'imbriquer les SELECT (par oppostion aux cas où il est suffisant, et préférable, d'utiliser WITH ou GROUP BY).

MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mffps — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Le Langage SQL
SELECT - Quantification générale (1)

- $\exists\ x.\ p(x)$:
  - *ensemble* ***non*** *vide*
  - 0 <> (SELECT count(*) FROM *ensemble*)
  - EXISTS (*ensemble*)
- $\neg\ \exists\ x.\ p(x)$:
  - *ensemble vide*
  - 0 = (SELECT count(*) FROM *ensemble*)
  - NOT EXISTS (*ensemble*)

- Pourquoi alors définir un nouvel opérateur ?
  - … plus simple, plus expressif et
  - … plus facilement détecté par le traducteur à des fins d'optimisation

2025-02-11 MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, M#ps — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

0 le nombre de tuple.
https://explainextended.com/2009/09/16/not-in-vs-not-exists-vs-left-join-is-null-postgresql/

Le Langage SQL
SELECT - Quantification générale (2)

- Quantification universelle absente ?
  - Il n'y pas d'opérateur « FORALL »
- Pas un problème, puisque
  - $\forall x.\ p(x) \equiv \neg\exists x.\ \neg p(x)$
- Donc
  - FORALL (cond) ≡
    NOT EXISTS (NOT cond)
  - FORALL (requête [cond]) ≡
    NOT EXISTS (requête [NOT cond])

MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mjftss — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Le Langage SQL
SELECT (syntaxe simplifiée) - Quantification

restriction ::=
[WHERE *condition*]
condition ::=
quantification | *expression* | ...

quantification ::=
[NOT] EXISTS (*requête*)

- En conséquence,
  - il y aura un emboitement de SELECT
  - le SELECT intérieur est lié au SELECT extérieur par les attributs provenant du FROM du SELECT extérieur et utilisés dans la condition du SELECT intérieur.
- Et plus, si affinités ☺

2025-02-11 MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, M#ps — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Exemple

```
select A.x
from A
where not exists (
  select 1
  from B
  where p(B.y,A.x)
)
```

{x ∈ A | [∀ y ∈ B] (not p(y,x) }

Remarques

1. En pratique, on utilise souvent un produit cartésien restreint sur le modèle de l'équivalence entre la jointure et le produit cartésien restreint.
2. En pratique, on ne s'intéresse le plus souvent qu'à la propriété dans un not exists(), d'où la sélection de la constante « 1 ».

Le Langage SQL
Quantification – exemple générique 1

```
select A.x
from A
where not exists (
  select 1
  from B
  where not p(A.x,B.y)
)
```

→ $\{x \in A \mid [\forall\, y \in B]\ (p(x,y))\}$

Remarque

1. En pratique, on utilise souvent un produit cartésien restreint sur le modèle de l'équivalence entre la jointure et le produit cartésien restreint.

2025-02-11 MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mffps — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Pourquoi « select 1 » ?
En fait, on peut utiliser n'importe quelle valeur, puisque seule l'existence importe (pas la valeur en tant que telle).

Le Langage SQL
Quantification – exemple générique 2

```
select A.x
from A
where not exists (
  select 1
  from B
  where A.x=B.x and not p(B.x,B.y)
)
```

→ $\{x \in A \mid [\forall (x,y) \in B] (p(x,y))\}$

Remarque

1. le produit cartésien est restreint par B (il aurait pu l'être par une condition totalement indépendante de A et B).

Le Langage SQL
Quantification – exemple générique 3

```
select A.x
from A
where q(A.x) and not exists (
  select 1
  from B
  where and not p(A.x,B.y)
)
```

→ $\{x \in A \mid q(x) \wedge [\forall\, y \in B]\ (p(x,y))\}$

Remarque

1. condition q est indépendante de B.

2025-02-11 MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mjftss — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Quantification – exemple générique 4

```
select A.x
from A
where q(A.x) and not exists (
  select 1
  from B
  where A.x=B.x and not p(B.x,B.y)
)
```

→ $\{x \in A \mid q(x) \wedge [\forall (x,y) \in B]\ (p(x,y))\}$

Remarque

1. le tout réuni.

MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mjftss — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Exemple de données

Étudiant

matricule	nom	adresse
15113150	Paul	>ᕕᖅᓂᑐᖅ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

TypeÉvaluation

code	description
IN	Examen intra
FI	Examen final
TP	Travail pratique
PR	Projet

Activité

sigle	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

Résultat

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20133	80
15112354	FI	IFT187	20123	78
15113150	TP	IFT159	20133	75
15112354	FI	GMQ103	20123	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

2025-02-11 MCED_SQL_03c : Expressions, sélections spécialisées (v3.12f) © 2018-2025, Mytis — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

On remarque l'omission de la dénotation des types des attributs.
C'est fréquemment le cas dans les représentations graphiques, afin de les alléger.
Cela ne porte pas à conséquence dans la mesure où une définition textuelle les accompagne, ce qui est le cas ici.

Nous verrons bientôt d'autres représentations graphiques plus complètes.

Note : >ᕕᖅᓂᑐᖅ se prononce (approximativement) Puvirnituq

Exemple de quantification 1

Quels sont les étudiants qui n'ont aucune évaluation à ce jour ?

L'ensemble recherché

S = { e ∈ Étudiant | e.matricule ∉ Résultat $\boldsymbol{\pi}$ {matricule} }

Le prédicat

[∀ e ∈ S] (e.matricule ∉ Résultat $\boldsymbol{\pi}$ {matricule})

MGBD_SQL_03c – Expressions: sélections spécialisées (v312f) © 2018–2025, Mητις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

{ e ∈ Étudiant | e ∉ Résultat $\boldsymbol{\pi}$ {matricule} }

Exemple de quantification 1

Quels sont les étudiants qui n'ont aucune évaluation à ce jour ?

```
SELECT *
FROM Etudiant
WHERE NOT EXISTS
  (
    SELECT 1
    FROM Resultat
    WHERE Etudiant.matricule =
            Resultat.matricule
  )
```

Note

« SELECT 1 » car, peu importe ce qu'on sélectionne, l'inexistence sera invalidée.

Étudiant

matricule	nom	adresse
15113150	Paul	ᐳᕕᖅᑐᖅ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Résultat

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20133	80
15112354	FI	IFT187	20123	78
15113150	TP	IFT159	20133	75
15112354	FI	GMQ103	20123	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

	matricule ▲ 1	nom	adresse
1	15113870	Mohamed	Tadoussac

MGBD_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mjhs — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec
2025-02-11

{ e ∈ Étudiant | e ∉ Résultat π {matricule} }

Exemple de quantification 2

Quelles sont les activités n'ayant aucun étudiant ?

```
SELECT *
FROM activite
WHERE NOT EXISTS
  (
    SELECT 'toto'
    FROM resultat
    WHERE resultat.activite =
          activite.sigle
  )
```

Activité

sigle	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

Résultat

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20133	80
15112354	FI	IFT187	20123	78
15113150	TP	IFT159	20133	75
15112354	FI	GMQ103	20123	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

	sigle	titre
1	IGE401	Gestion de projets

Exemple de quantification 3

Quels sont les étudiants sans résultats en 2013 ?

```
SELECT *
FROM Etudiant
WHERE NOT EXISTS
 (
 SELECT matricule
 FROM Resultat
 WHERE Etudiant.matricule =
          Resultat.matricule
   AND NOT
    (trimestre BETWEEN
     '20131' and '20133')
 )
```

FALSO!

Étudiant

matricule	nom	adresse
15113150	Paul	>ᕕᖕᓂᑐᖅ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Résultat

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20133	80
15112354	FI	IFT187	20123	78
15113150	TP	IFT159	20133	75
15112354	FI	GMQ103	20123	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

matricule	nom	adresse
15113150	Paul	>ᕕᖕᓂᑐᖅ
15113870	Mohamed	Tadoussac

2025-02-11 MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mᕕᖕᓂᑐᖅ — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

On obtient les étudiants dont TOUS les résultats sont en 2013!
(y compris Mohamed qui n'a… acun réultat, bien sûr)

Exemple de quantification 3 (bis)

Quels sont les étudiants sans résultats en 2013 ?

```
SELECT *
FROM Etudiant
WHERE NOT EXISTS
 (
 SELECT matricule
 FROM Resultat
 WHERE Etudiant.matricule =
          Resultat.matricule
   AND
    (trimestre BETWEEN
     '20131' and '20133')
 )
```

JUSTO!

Étudiant

matricule	nom	adresse
15113150	Paul	>ᕕᖕᓂᑐᖅ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Résultat

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20133	80
15112354	FI	IFT187	20123	78
15113150	TP	IFT159	20133	75
15112354	FI	GMQ103	20123	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

	matricule	nom	adresse
1	15112354	Éliane	Blanc-Sablon
2	15113870	Mohamed	Tadoussac

On obtient les étudiants la bonne réponse.
(y compris Mohamed qui n'a… aucun résultat en 2013, bien sûr)

SELECT - Quantification de comparaison

- Des raccourcis simples et efficients ciblant les opérateurs relationnel simples :
  - ∃ *x* ∈ *ensemble*. *expression opComp x*
    - se traduit en SQL par : *expression opComp* ANY *ensemble*
  - ∀ *x* ∈ *ensemble*. *expression opComp x*
    - se traduit en SQL par : *expression opComp* ALL *ensemble*
  - *opComp* ::=
    = | <> | < | <= | > | >=
  - *ensemble* ::=
    (*requête* **)** | **(** *listeDeValeurs* **)**

Exemple de quantification 4

Quels sont les étudiants dont toutes les notes sont supérieures ou égales à 60 ?

```
SELECT matricule
FROM Etudiant
WHERE 60 <= ALL
  (
  SELECT note
  FROM Resultat
  WHERE Etudiant.matricule =
    Resultat.matricule
  )
```

Étudiant

matricule	nom	adresse
15113150	Paul	>ᕕᖅᑐᖅ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Résultat

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20133	80
15112354	FI	IFT187	20123	78
15113150	TP	IFT159	20133	75
15112354	FI	GMQ103	20123	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

matricule	nom	adresse
15113150	Paul	>ᕕᖅᑐᖅ
15113870	Mohamed	Tadoussac

2025-02-11 MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

On obtient la bonne réponse.
(y compris Mohamed qui n'a… aucun résultat en 2013, bien sûr)

ce que ne donnerait pas

```
SELECT matricule
FROM Resultat
WHERE 60 <= ALL
  (
  SELECT note
  FROM Resultat
  WHERE Etudiant.matricule =
  Resultat.matricule
  )
```

Serait-mieux avec les windows function ? ☺

Ordonnancement

Le Langage SQL
SELECT (syntaxe simplifiée) - Ordonnancement

Ordonnancement ::=

[ORDER BY { *expression* [*ordre*] [*t_null*] ... , }]
[LIMIT { nombre | ALL }]
[OFFSET nombre]

order :: =

ASC | DESC | USING *opérateur*

t_null ::=

NULLS { FIRST | LAST }

MCED_SQL_03c : Expressions, sélections spécialisées (v312f) © 2018-2025, Mffps — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

[ORDER BY *expression* [ASC | DESC | USING *opérateur*] [NULLS { FIRST | LAST }] [, ...]]
[LIMIT { *nombre* | ALL }]
[OFFSET *nombre*] [ROW | ROWS]]
[FETCH { FIRST | NEXT } [*total*] { ROW | ROWS } ONLY]
[FOR { UPDATE | SHARE } [OF *nom_table* [, ...]] [NOWAIT] [...]]

LIMIT n : les n premiers tuples sont extraits selon l'ordre spécifié
OFFSET n : tous les tuples sont extraits sauf les n premiers selon l'ordre spécifié (le complément de LIMIT, si exclusif)

Qu'arrive-t-il si les deux sont présents ?
Sont-ils appliqués dans l'ordre ou l'un a-t-il préséance sur l'autre ?

Références

Références

- Elmasri et Navathe (4e ed.), chapitre 7
- Elmasri et Navathe (6e ed.), chapitre 4
- [Loney2008]
  Loney, Kevin ;
  Oracle Database 11g: The Complete Reference.
  Oracle Press/McGraw-Hill/Osborne, 2008.
  ISBN 978-0071598750.
- [Date2012]
  Date, Chris J. ;
  SQL and Relational Theory : How to Write Accurate SQL Code.
  2nd edition, O'Reilly, 2012.
  ISBN 978-1-449-31640-2.
- Le site d'Oracle (en anglais)
  - http://docs.oracle.com/cd/E11882_01/index.htm
- Le site de PostgreSQL (en français)
  - http://docs.postgresqlfr.org
  - plus particulièrement la section 9.21
    https://doc.postgresql.fr/15/functions-aggregate.html